Doomscroll -- Lab 3 Design Model

Refines the Lab 2 conceptual model (conceptual_model.pdf) per Lab 3 Section 3.1. Lab 2's 24 classes are carried forward; this revision folds in the ML training pipeline that has since been built (not anticipated in Lab 2 -- see ../docs/ML_PIPELINE.md and ../src/doomscroll/ml/) and splits the single GazeEstimationController into a Strategy-pattern structure that matches what the code actually does: it already supports two interchangeable gaze-prediction methods (calibrate.py's linear calibration, and the newer trained ML model), selected at startup by scripts/phase2_gaze_scroll.py:make_predictor().

Status tags below ([BUILT] / [PLANNED]) say what's implemented today vs. still designed-for. Per the Lab 3 manual, submitting an in-progress prototype is acceptable as long as the skeleton structure is clear -- rather than deleting unbuilt classes, we keep them with honest status so requirements traceability back to Lab 1/2 isn't lost.

1. What changed since Lab 2, and why

1. No GUI framework was adopted. Lab 2's tech stack discussion landed on PySide over the AI's Tauri recommendation (see tech_stack_report.pdf, Section 3), but implementation so far is plain Python scripts with an OpenCV debug window (src/doomscroll/debug/overlay.py) -- no SetupScreen/CalibrationScreen/SettingsScreen have been built as real GUI windows yet. We keep these boundary classes in the design (still the plan for Lab 4) but mark them [PLANNED], with the live cv2 overlay standing in as a temporary ReaderOverlay.
2. Content-awareness was not built. ContentAnalysisController, ContentElement, and ImportantContentRegion remain [PLANNED] -- the current trigger is a fixed "bottom 15% of the focused window" zone (fixation/attention_gate.py), not actual content classification.
3. A substantial ML pipeline was built that Lab 2 never modeled: recording real webcam+cursor sessions, extracting features, training/comparing regression models (ridge / MLP / gradient boosting), and applying the trained model live. This is new design work, captured in Sections 3-4 below.
4. Two new real boundary classes exist that Lab 2's model didn't need yet: FaceLandmarkBackend (wraps the MediaPipe face-mesh model) and WindowBoundsProvider (platform-specific focused-window bounds, needed for the dwell-trigger zone -- distinct from ScreenCaptureInterface, which reads pixel content, not window geometry).

2. Updated Class List

Unchanged Lab 2 classes are not re-described here (see conceptual_model.pdf); only additions, splits, and status changes are listed.

Boundary

Class | Status | Notes
`AppScreen` (abstract), `SetupScreen`, `CalibrationScreen`, `SettingsScreen`, `RecalibrationPromptDialog` | `[PLANNED]` | Unchanged from Lab 2; no GUI framework chosen/built yet -- see Section 1.
`ReaderOverlay` | `[BUILT, simplified]` | Implemented today as a `cv2.imshow` debug window (`debug/overlay.py`: `draw_gaze_minimap`, `draw_status_lines`, `draw_iris_points`) -- functional but not the always-on-top, click-through HUD the mockups describe.
`WebcamInterface` | `[BUILT]` | `capture/webcam.py: WebcamSource`.

`ScreenCaptureInterface` | `[PLANNED]` | Needed once `ContentAnalysisController` is built; `mss` is already a project dependency for this.
`OSScrollAPI` | `[BUILT]` | `scroll/`: platform-specific injectors behind `get_scroll_injector()`.
`FaceLandmarkBackend` *(new)* | `[BUILT]` | `landmarks/mediapipe_backend.py: MediaPipeLandmarkBackend` -- boundary to the MediaPipe face-mesh model; returns 478-point landmarks per frame.
`WindowBoundsProvider` *(new)* | `[BUILT]` | `windowing/`: `get_window_bounds_provider()` -- boundary to the OS's focused-window geometry (Win32/Cocoa), used to compute the dwell-trigger zone. Distinct from `ScreenCaptureInterface`.

Control

Class | Status | Notes
`CalibrationController` | `[BUILT, split]` | `scripts/calibrate.py` (16-point) and `scripts/quick_calibrate.py` (9-point `AffineCorrection` fit) are two concrete flows of this control class -- see Strategy/pattern note below.
`GazeEstimationController` | `[BUILT, refactored into Strategy]` | See Section 3 -- now a thin coordinator over a pluggable `GazePredictionStrategy`, plus the always-applied `HeadPoseEstimator` and `GazePointSmoother`.
`ContentAnalysisController` | `[PLANNED]` | Unchanged from Lab 2; not yet implemented.
`AutoscrollController` | `[BUILT, simplified]` | Implemented as the dwell-trigger logic in `scripts/phase2_gaze_scroll.py` (`AttentionGate` + bottom-dwell-fraction check), not yet a standalone class -- flagged as a Lab 4 refactor target (see Section 5).
`SettingsController` | `[PLANNED]` | No `AutoscrollSettings` persistence exists yet; CLI flags stand in for configuration today.
`ModelTrainingController` *(new)* | `[BUILT]` | `scripts/train_model.py` + `src/doomscroll/ml/training.py` -- offline control class: compares candidate models (`ridge`/`mlp`/`hgb`) via leave-sessions-out cross-validation, selects and persists the best one. Not part of the live app's runtime control flow; a tooling-side control class.
`DatasetBuilderController` *(new)* | `[BUILT]` | `scripts/build_dataset.py` + `src/doomscroll/ml/dataset.py` -- turns a recorded `Session` into labeled training frames (runs `FaceLandmarkBackend` + feature extraction over every frame of a recording).

Entity

Class | Status | Notes
`User`, `AutoscrollSettings`, `Session` | `[PLANNED]` / `[BUILT, no persistence]` | `Session` exists as recorded data (`recording/session.py: SessionMeta`) but is training data, not the Lab 2 "one period of app usage" concept -- see note in Section 3.
`CalibrationProfile` | `[BUILT, two forms]` | `gaze/calibration.py` (linear, old path) and `ml/model.py: AffineCorrection` (new path's per-user correction) -- see Strategy pattern, Section 3.
`HeadPose`, `GazeVector`, `GazePoint` | `[BUILT]` | `gaze/head_pose.py: HeadPose`; `gaze/gaze_vector.py`; both consumed into `ml/features.py`'s 11-dim feature vector.
`ContentElement`, `ImportantContentRegion` | `[PLANNED]` | Unchanged from Lab 2.
`ScrollCommand` | `[BUILT, implicit]` | Not a persisted/explicit object yet -- `scroll/` injectors are called directly rather than constructing a `ScrollCommand` value object first. Noted as a Lab 4 refactor opportunity (Section 5).
`FeatureVector` *(new)* | `[BUILT]` | `ml/features.py: FEATURE_NAMES` -- the 11-number vector (gaze offsets, eyelid ratio, head yaw/pitch/roll, head translation) that both training and live inference consume. Single source of truth: `extract_features()` is shared by both paths so they cannot drift apart.
`TrainedGazeModel` *(new)* | `[BUILT]` | `ml/model.py` -- a persisted `(estimator, metadata, AffineCorrection?)` bundle; `predict_px()` is the runtime interface `GazeEstimationController` calls.
`LabeledFrames` / `SessionDataset` *(new)* | `[BUILT]` | `ml/dataset.py` -- per-session feature/label arrays; `LabelSettings` (lag, idle-filter, click-mode) parameterizes how raw recordings become training labels.

3. Design Pattern: Strategy for gaze estimation

The clearest real pattern already present in the code, now formalized in the design:

```
«interface»
GazePredictionStrategy
  + predict(points_px, head_pose, frame_gray) -> GazePoint

LinearCalibrationStrategy          MLModelStrategy
  (wraps GazeCalibrator,             (wraps TrainedGazeModel,
   gaze/calibration.py)               ml/model.py)
```

GazeEstimationController holds a GazePredictionStrategy (chosen at startup by --model flag -- make_predictor() in phase2_gaze_scroll.py) and always applies the same downstream pipeline regardless of which strategy is active: HeadPoseEstimator (stateful PnP pose, seeded frame-to-frame) -> strategy's raw prediction -> GazePointSmoother (Kalman filter with innovation gating -- see gaze/smoothing.py) -> the dwell-trigger logic. This is why the two "calibration profile" entities above (linear calibration's params vs. AffineCorrection) are drawn as two concrete forms of one design-level CalibrationProfile role, not duplicated classes -- the Strategy pattern is exactly what lets both coexist cleanly, satisfying "loose coupling" (Lab 3 Section 2.3): AutoscrollController and ReaderOverlay depend only on the GazePoint the controller produces, never on which strategy produced it.

This also directly answers Lab 3 Section 3.2 (persistent data / access control): the one piece of genuinely sensitive persistent data in this design is raw webcam video (data/recordings/, explicitly git-ignored, never uploaded -- see docs/ML_PIPELINE.md). Everything downstream (FeatureVector, TrainedGazeModel) is derived, non-biometric, and safe to share/commit; models/gaze_model.joblib is committed to the repo for exactly this reason.

4. Design Pattern: pipeline / single-responsibility split in the ML path

DatasetBuilderController -> ModelTrainingController -> TrainedGazeModel mirrors a build-pipeline pattern, deliberately kept as separate control classes rather than one large class (see docs/ML_PIPELINE.md's own rationale table) -- each stage is independently testable and only the slow stage (MediaPipe feature extraction) needs re-running when recordings change; label parameters (LabelSettings: lag, idle-filter, click-mode) can be swept at training time without re-running feature extraction, which is a direct instance of high cohesion / low coupling (Lab 3 Section 2.3) between "what a frame looks like" and "which frames are trustworthy labels."

5. Known gaps to close in Lab 4 (traceability notes)

- AutoscrollController and ScrollCommand should become real classes (currently inline logic + direct injector calls) once ContentAnalysisController lands, so scroll decisions can depend on both gaze and content boundary as Lab 2's use case 4.1 specifies.
- SettingsController / AutoscrollSettings persistence doesn't exist; CLI flags are a placeholder. This blocks Use Case 5.1 from being demoable as designed.
- The boundary/GUI layer (SetupScreen etc.) is entirely unbuilt -- the single biggest gap between the Lab 2 design and Lab 3 reality, and the subject of the re-prompted AI tech-stack exercise below (Section 3.3).